Unit 2 - Relational Databases

Entity-Relationship model and diagram design (part 1)

Introduction

The Entity-Relationship (ER) model is a high-level conceptual data model used for designing databases.

It provides an abstract and easy-to-discuss representation of data, focusing on the **entities** (real-world objects or concepts), their **attributes** (properties), and the **relationships** between those entities.



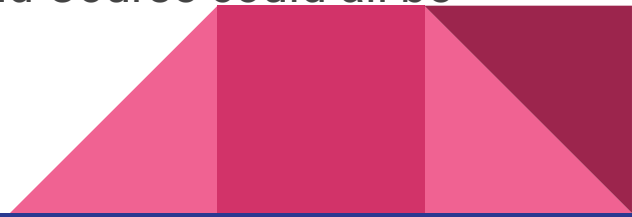
Entities

In ER modeling, an entity is a distinguishable object or concept in the real world about which we need to store data.

An entity can be a tangible object (e.g. Student, Car, Employee) or an intangible concept (e.g. Course, Job Position).

It represents a **noun** in our data model – similar to how nouns represent people, places, or things in language.

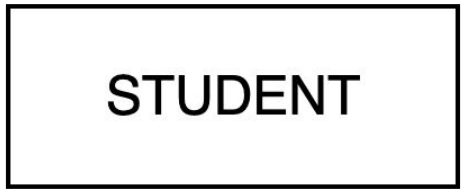
For example, in a school database, Student, Professor, and Course could all be entities.



Entities

We usually represent an entity type as a rectangle labeled with the entity name in ER diagrams.

For instance, a rectangle labeled STUDENT represents the entity type “Student” and stands for the entire set of students in the database



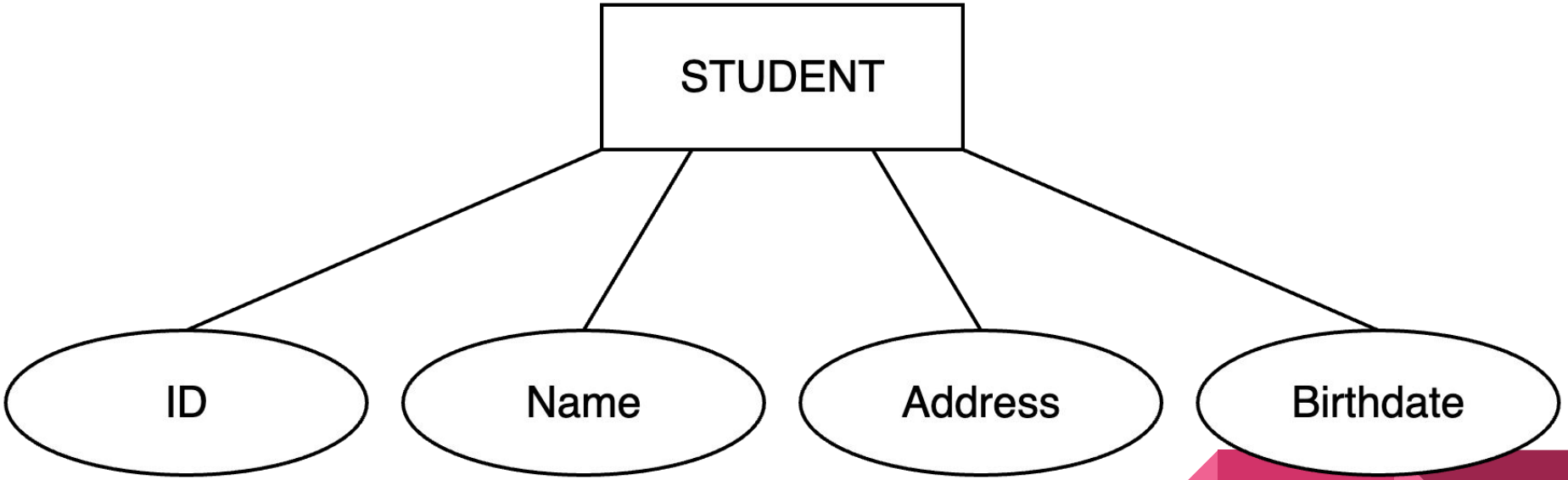
Attributes

Attributes are the properties or characteristics that describe an entity. In a database, each attribute becomes a field (column) stored for the entity.

For example, a Student entity might have attributes like student ID, name, birthdate, and address. In ER diagrams, attributes are typically shown as ovals connected to their entity (rectangle) by a line.

Each oval is labeled with the attribute name.

Attributes



Entities and attributes

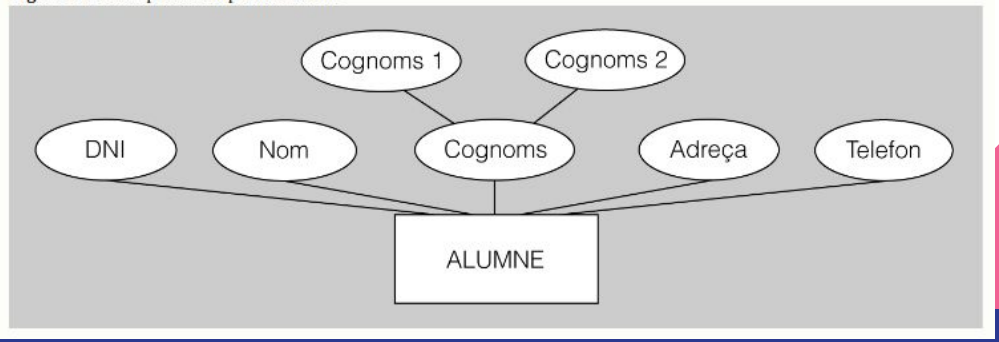
An attribute of an entity **may have no value**. In these cases, the attribute is also said to have NULL value .

A **simple** attribute can not be divided into smaller parts without this entailing the loss of its meaning.

A **composite** attribute is one that is subdivided into smaller parts (which are also considered attributes), which have their own meaning.

Ex. Surnames

Figure 1.2. Example of composite attribute



Entities and attributes

A **derived** attribute when its value can be calculated from other attributes or other entities interrelated.

A specific instance entity must be able to be distinguished from other real-world objects. Therefore, any ER modeling must indicate, for any type entity, an attribute or set of attributes that allows it to be **uniquely identified**.

The attribute or set of attributes that uniquely identify entities are called **primary key**.

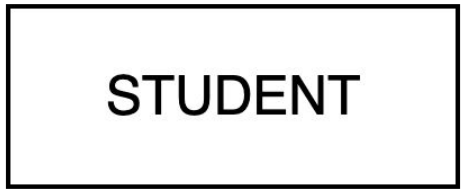
In ER notation, key attributes are usually underlined or noted with “PK”.



Notation - Entities

In this representation we will respect the following characteristics:

- As a general rule, we will **not use accents or special characters**, only letters and numbers.
- We will represent the type **entities** by writing their name in **capital letters** and in the **singular**, inside a **rectangle**.



Notation - Attributes

In this representation we will respect the following characteristics:

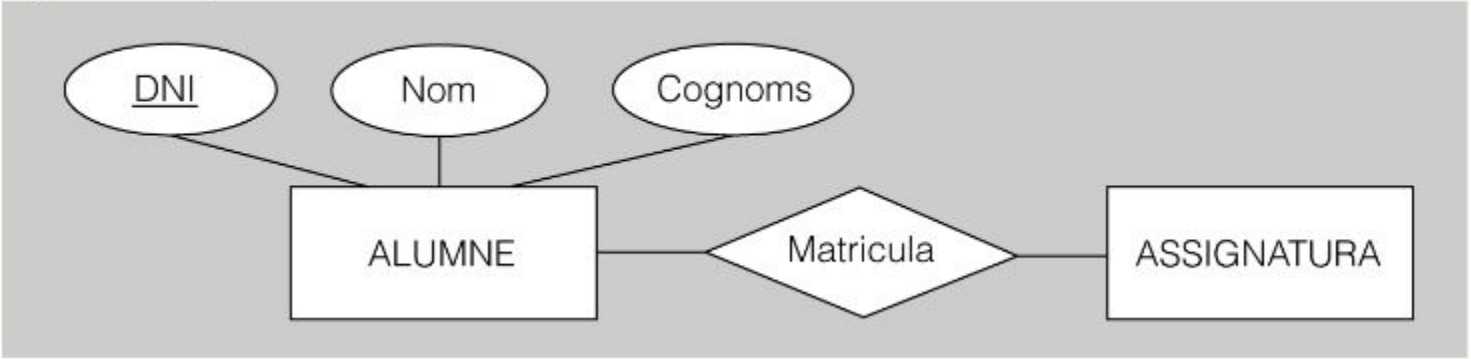
- We will represent each **attribute** writing its name with the **first letter in capital letters** and the rest in small letters, **within an ellipse** united with a line with the rectangle that represents the entity of which they comprise:
  - If an attribute has a **compound name**, **each name will start with a capital letter** to make it more readable. For example, TelefonFix, TelefonMobil. This is known as "*PascalCase*".
  - If the name of an **attribute** corresponds to an **acronym**, it must be in **capital letters**, such as DNI (national identity document).



Notation - Relationships

An **relationship** is an association between two or more entities. They are represented by a rhombus (or “diamond” shape).

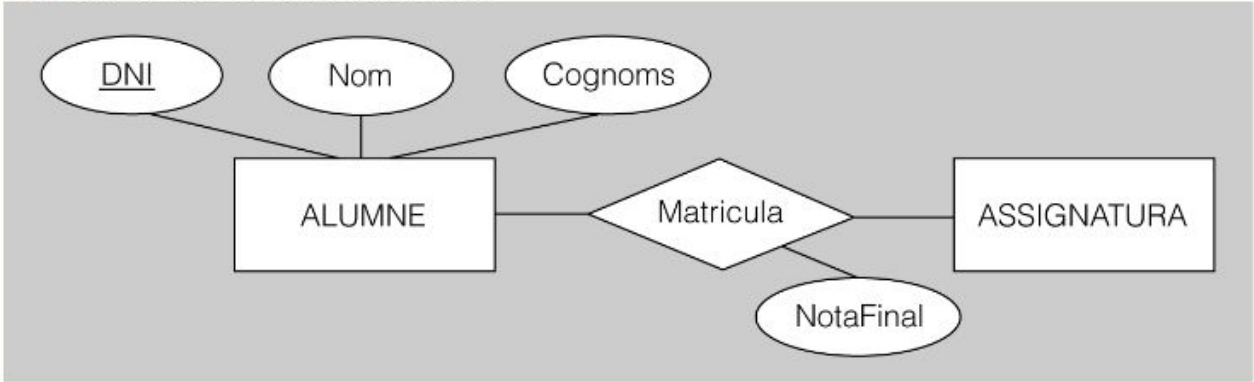
Figure 1.6. Example of interrelation



Relationships

Sometimes, we may be interested in reflecting on some **characteristics of certain interrelationships**. The way to do this is to add the necessary attributes, as we would if we worked with entities. These attributes are the **attributes of the relationship**.

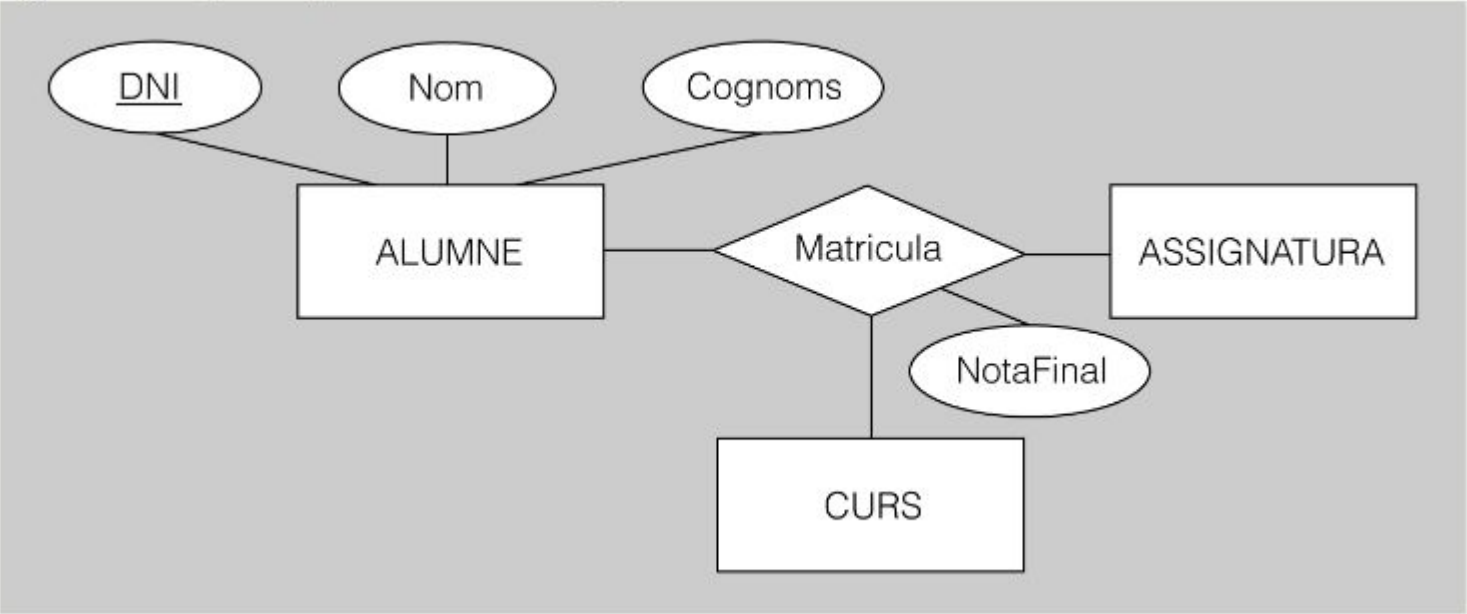
Figure 1.7. Example of interrelationship attribute



Relationships

The grade of a relationship depends on the number of entities it associates with.

Figure 1.8. Example of degree three interrelationship



Relationships - Cardinality

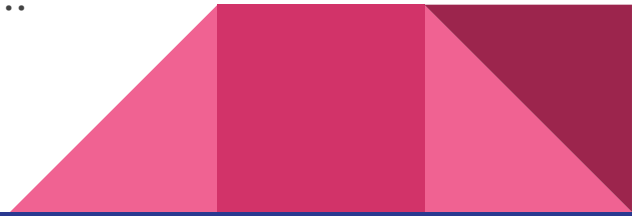
The cardinality indicates the interrelation of correspondence between occurrences of entities to associate itself.

Binary interrelationships (2 entities, grade 2 relation) can offer three types of connectivity:

- One on one (1: 1)
- One to many (1: N) or Many to One (N: 1)
- Many to many (N: M)

You can add a 0 when the relation is optional (0..1, 0..N) ...

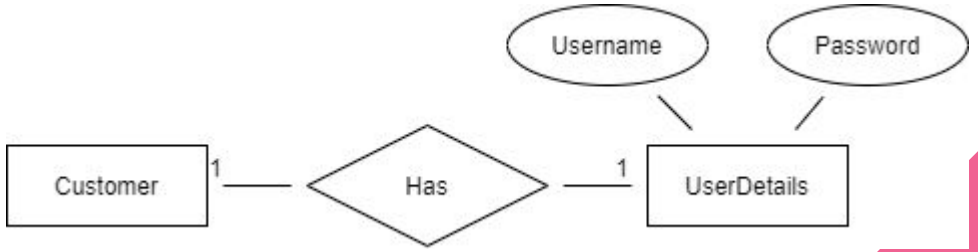
... and a 1 for making it mandatory (1..1, 1.. N)



Cardinality - One to one (1: 1)

A one-to-one relationship is represented by a line without any annotations that joins two entities.

One-to-one means that for the two entities connected by the line, there is exactly one instance of the first entity for each one instance of the second entity. An example might be customers and user details: each customer has exactly one username and password, and that particular username and password is only for that customer.



Cardinality - One-to-many (1 : N)

A one-to-many relationship is represented by a line annotated with a 1 and an N (or a 1 and an M).

One-to-many means that for the two entities connected by the line, there are one or more instances of the second entity for each one instance of the first entity. From the perspective of the second entity, any instance of the second entity is related to only one instance of the first entity. An example is wineries and wines: each winery sells many wines, but each wine is made by exactly one winery. Many-to-one relationships are the most common relationships between entities.



Cardinality - Many-to-many (N: M)

A many-to-many relationship is represented by a line annotated with an M and an N.

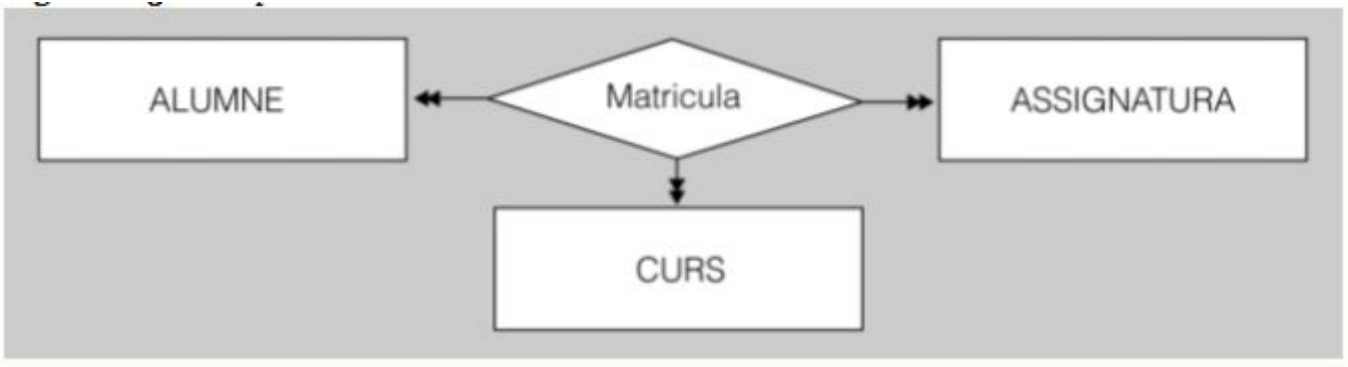
Many-to-many means that for the two entities connected by the line, each instance of the first entity is related to one or more instances of the second entity and, from the other perspective, each instance of the second entity is related to one or more instances of the first entity. An example is the relationship between wineries and delivery firms: a winery may use many delivery firms to freight wine to customers, while a delivery firm may work for many different wineries.



Relationships - Ternary

Ternary interrelationships (grade 3) can offer four types of connectivity:

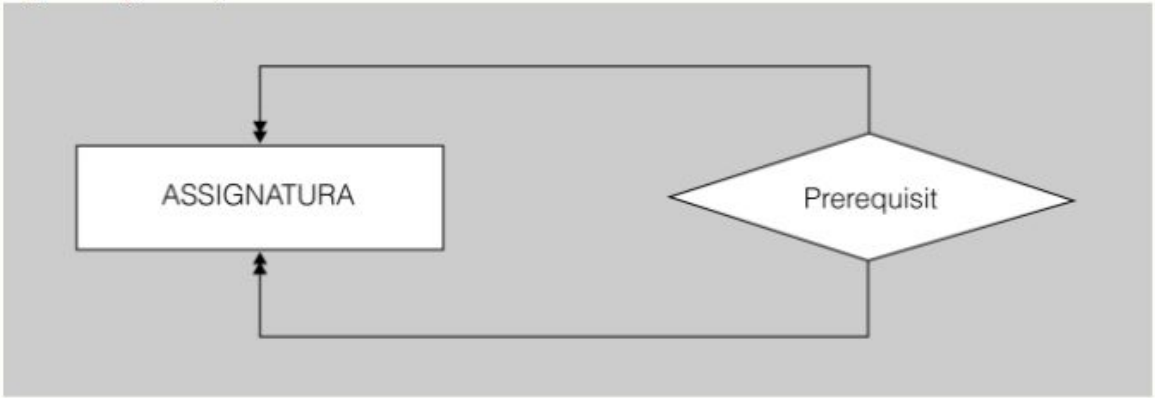
- 1:1:1
- 1:1:N
- 1:M:N
- M:N:P



Relationships - Recursive

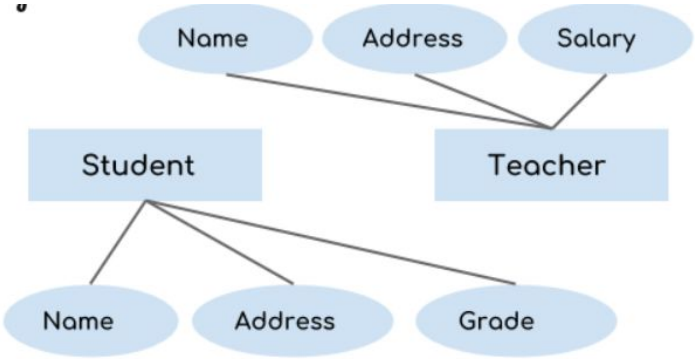
A recursive interrelationship associates instances of one entity with other instances of the same entity.

A recursive interrelationship is said to be grade 1 (or unary) if only one entity participates in it, which relates to itself.

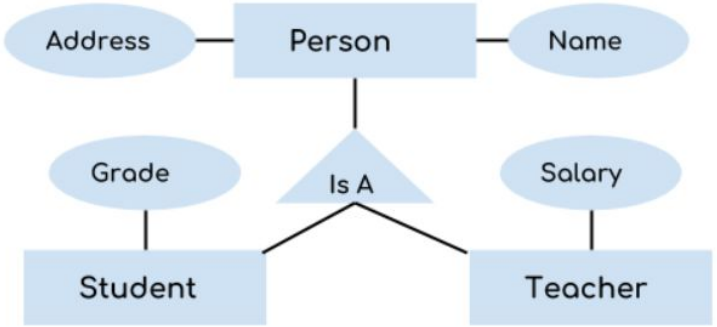


Inheritance

Inheritance can be described as a method used to create a general entity by picking all the common attributes from other entities in the Database Management System. Is represented with an inverted triangle.



Before Generalization



Generalization

To sum up...

Rectangles

Represent entities—that is, objects being modeled. Each entity is labeled with a meaningful title.

Diamonds

Represent relationships between entities; a relationship is labeled with a descriptive title that represents how the entities interact.

Ellipses

Represent attributes that describe an entity.

Lines between rectangles and diamonds

Connect entities to relationships. Lines may be without any annotation, be annotated with an M and an N, or annotated with an M and a 1 (or an N and a 1). Annotations indicate the cardinality of the relationship.

Lines between rectangles and ellipses

Connect attributes to entities. These lines are never labeled.



Participation (min,max)

Defines the maximum and minimum participation within a relationship. Not the same as cardinality, but closely related.

1:1 → describes the relationship type/cardinality: one entity instance relates to one instance of the other.

(1,1) → describes the minimum and maximum participation: at least 1 and at most 1.

For example:

Student — 1:1 — Locker → relationship is one-to-one.

Student (0,1) — (1,1) Locker → specifies the participation constraints: a student may have 0 or 1 locker, while every locker belongs to exactly 1 student.

So 1:1 does not tell you whether participation is mandatory or optional; (min,max) does.

Chen vs Crow's foot

Chen Notation	Crow's Foot	Explanation
1 (1,1)		One
1 (1,1)		Only One
(0,1)		Zero or One
M		Zero or More
M (1,M)		1 or More

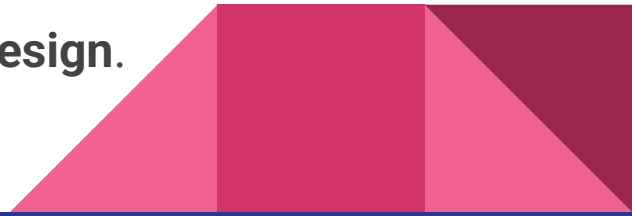
Entity-relationship diagrams

Entity-Relationship diagrams are key graphical tools in database design. Its preparation must be systematic and rigorous if a valid and efficient system is to be obtained.

The design of DB is structured, fundamentally, in three main stages:

1. Conceptual design
2. Logical design
3. Physical design

Here we focus on the study and practice of **conceptual design**.

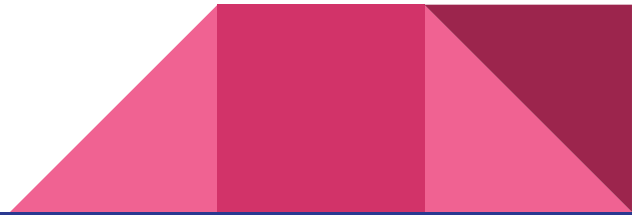


Conceptual design

The first thing to do, during the conceptual design phase, is to gather all the necessary information from the part of the real world that we intend to model with a DB.

This collection of information will be done in different ways, such as these:

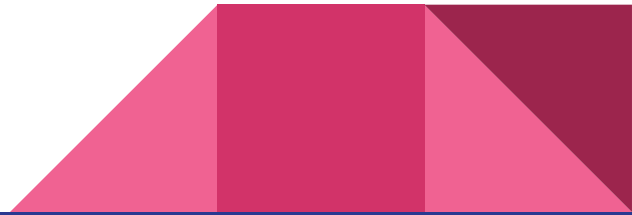
- Interviews with future users of the DB being designed.
- Examination of the documentation provided by these same users.
- Direct observation of the processes to be computerized.



Conceptual design

Finally, it is necessary to choose a high-level data model. In our case, the high-level data model used to begin with is the *Entity-Relationship*.

Expressed in the terminology of the *ER* model, the data schema developed during the conceptual design phase must specify all the necessary **entities**, and the inter**relationships** between them, with the appropriate **cardinalities**, as well as the corresponding **attributes** in each case.



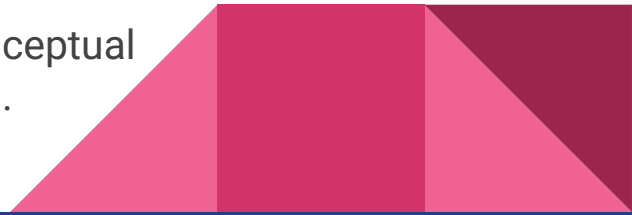
Logical design

In the logical design phase , we work with the abstract data model obtained at the end of the conceptual design stage, in order to translate it into the data model used by the database management system (DBMS) with which you want to implement and maintain the database.

It is necessary to take into account the specific technology to be used in the creation of the DB, as the resulting DB can be adapted to different logical models, such as the following:

- Hierarchical
- **Relational**
- Distributed
- Object oriented

The most common, when designing a DB, is still to express the conceptual scheme in an ER model and then translate it into a relational model.



Physical design

The physical design phase is characterized by the possibility of making other decisions, related to aspects of physical implementation at a lower level, and closely linked to the DBMS with which we work in each case, such as the following:

- Definition of indexes.
- Allocation of the initial space for the tables, and forecast of their further growth.
- Selection of buffer size.
- Parameterization of the DBMS according to the options it offers.

It is necessary to take into account the characteristics of the processes that operate with the data, the frequency of execution of the different types of queries, the degree of volatility of the data, the volumes of information to be stored. , etc.